

FINAL PROJECT PART 3

Ranking and Filtering

Laura NARANJO & Laura PEÑALVER & Aurora PUJOLS

November 20, 2025

TAG: IRWA-2025-part-3
GitHub: <https://github.com/aurorapujols/irwa-search-engine>

We separate this part 3 of the project in three parts:

- a. **Rankings (1)** in which we create three different ways of ranking (TF-IDF+cosine similarity, BM25, and a custom score).
 - b. **Word2Vec (2)**: in which we implement the word2vec + cosine ranking score.
 - c. **Engine (3)**: in which we explain the engine we created to perform the different rankings.
- All the functions and explanations are also in the Jupyter Notebook called `ranking_and_filtering.ipynb`.

1 PART 1: Rankings

AI use

CODE: used ChatGPT and Copilot to help in the debugging process.

In this part, we were asked to compute the ranking of products using three different algorithms.

1.1 TF-IDF + Cosine Similarity

For the ranking computation using the **TF-IDF** and **cosine similarity**, we reused the code from the last part of the project (PART 2) as we already used TF-IDF ranking. But we had to change the way the score was computed because it was only using the dot product between the query vectors and the product vectors. To make it the cosine similarity we only needed to add the normalization by the norm of the product vector. We do not need to normalize by the query norm because it is the same for all products and therefore its a constant.

```
product_scores = []
for pid, current_prod_vec in product_vectors.items():
    doc_norm = np.linalg.norm(current_prod_vec)
    if doc_norm == 0: #avoid division by 0
        score = 0.0
    else:
        score = np.dot(current_prod_vec, query_vector) / doc_norm
    product_scores.append([score, pid])
# Sort by score descending
product_scores.sort(key=lambda x: x[0], reverse=True)
```

The previous code shows the only part of the code that was changed from the previous part. The function that computes the ranking is called `rank_tf_idf_cosine_similarity`.

1.2 BM25

For this ranking algorithm we will assume the following:

- The queries are not gonna be long (at most between 5-10 words).
- We do not have any relevance information on the documents.
- We will use values $k_1 = 1.2$ and $b = 0.75$ as tuning parameters.

The BM25 algorithm has as ranking function (with the previous assumptions) that attributes a RSV_d to document d as:

$$RSV_d = \sum_{t \in q} \log \frac{N}{df_t} \frac{(k_1 + 1)tf_{td}}{k_1((1 - b) + b \times (L_d/L_{ave})) + tf_{td}} = \sum_{t \in q} idf_t \frac{(k_1 + 1)tf_{td}}{k_1((1 - b) + b \times (L_d/L_{ave})) + tf_{td}}$$

- df_t : document frequency of term t
- tf_{td} : raw frequency of term t in document d
- L_d : length of document d
- L_{ave} : average document length in the whole collection
- k_1 : tuning parameter controlling the document term frequency scaling
- b : tuning parameter controlling the scaling by document length

In the TF-IDF ranking, we previously needed to compute the inverted index as well as idf , tf , and df . But because we used a special inverted index including the fields as sub-indices, we are going to calculate the data we need for this algorithm again so that we have it in the following format:

```
idf = {"term1": idf1, "term2": idf2, ...}
df = {"term1": tf1, "term2": tf2, ...}
Ld = {"pid1": L1, "pid2": L2, ...}
Lave = # average of all lengths in Ld
```

The previous variables are computed in the function `compute_prod_data`. For the term frequency `tf`, we compute it directly in the ranking only for the documents/products that will be ranked:

```
for pid in filtered_products:
    tf[pid] = Counter(products[pid])
# where products is a dict of pid -> ["term1", "term", ...] of all terms in the
# document/product pid
```

The ranking is done in the function `rank_BM25`.

1.3 Custom Ranking

For the custom ranking, we made use of the hint and thought about including the numerical values of each product in the score. Using the same assumptions as in the BM25 ranking algorithm, we construct the following ranking function to compute the score for a document d :

$$score_d = [\lambda RSV_d + (1 - \lambda)(\alpha R_{score} + \beta(D_{score} + P_{score}))]A_{score}$$

- λ : tuning parameter of the BM25 score, the higher the more the BM25 score is taken into account
- RSV_d : BM25 score (computed as in 1.2)
- α : **popularity**, weight of the product's rating in the final score
- β : **affordability**, weight of the product's price and discount in the final score
- R_{score} : **rating score** of the product computed as $\frac{rating}{5.00}$
- D_{score} : **discount score** of the product computed as $\frac{discount}{100}$
- P_{score} : **price score** of the product computed as $\frac{1}{1 + \log price}$

- A_{score} : **availability score** of the product that it is either a value close to zero if the product is out of stock, or 1 if the product is in stock

The reasoning behind the score is the following: We want to take into account the previously mentioned characteristics of the documents such as the term frequency, or idf values. Because BM25 is more robust than TF-IDF, we decided to use that score and include it in our own. But we want to take it into account only by a certain amount as we also want to include information about the numerical fields of our products in the score. That is why we use $\lambda \in [0, 1]$.

The part of the score that is not from the BM25 ranking is computed so that we consider the `rating` of a product with a certain amount α , where $0 \leq \alpha \leq 1$, and the discount and price of a product with an amount β , where $0 \leq \beta \leq 1$, with $\alpha + \beta = 1$.

The value α multiplies the rating divided by 5, as the ratings are from 0 to 5 and we wanted to normalize it for R_{score} . Then, the `selling_price` of a product is taken into account but computed into P_{score} so that a product with a lower price has a higher score (normalized similarly as the idf). The `discount` of a product is taken into account in D_{score} to give higher score to a product with higher discount. Finally, β is multiplied by the sum of D_{score} and P_{score} as a single measure of price per product.

Finally, A_{score} multiplies the weighted sum of the BM25 score and the custom score by a value close to zero if the product is `out_of_stock`, with the aim of ranking them at the last positions, but still sorting them (not last in various positions with same score equal to 0.0).

As standard values, we chose the following: $\lambda = 0.3$, $\alpha = 0.3$, $\beta = 0.7$, and `availability` = 0.01.

The ranking is done in the function `rank_custom`. Additionally, we needed the function `get_numerical_info_by_product` to pre-process the numerical values of each product (basically imputing missing values).

1.4 Ranking Algorithms' Review

We can first compare the three rankings obtained with the same query. In figures 1, 2, and 3 we can see the results of the search for the query "**western leather jacket men**" for the algorithms TF-IDF + cosine similarity, BM25, and custom ranking, respectively.

In them we can see things like documents being ranked in completely opposite parts of the ranking (due to the extra tf and document length normalization in BM25), and the penalization for being out of stock in the last three products of the custom ranking. Also, some products are have different rank in the BM25 ranking and the custom ranking due to the price or the ranking of the products.

The most different rankings are TF-IDF and BM25. The differences are due to the way the rankings treat the term frequency and document length:

- TF-IDF takes into account the TF and IDF in the vector representations of both the query and the document with additional normalization of the TF to not automatically give higher scores to long documents that are more likely to have higher term frequencies. However, it does not take into account the fact that the term frequency should saturate (after a certain point, repeating the same term many times more should not increase the score linearly). In addition, it does not adjust the score directly normalizing it by document length.
- BM25 is a probabilistic retrieval model that was specifically designed to address the limitations of TF-IDF. The two tuning parameters k_1 and b control how the term frequency contributes to the score, and normalizes document length, respectively. Therefore, BM25 is considered to be more robust and effective for IR retrieval tasks.

Then, we can also look at the pros and cons of each of the three methods. They are written in table 1.

2 PART 2: Word2Vec

AI use

WORD2VEC MODEL: we asked ChatGPT references and ideas on what the parameters for wordvec model should be.

CODE: we used ChatGPT and Copilot to help in the debugging process

In this part, we describe the ranking with Word2Vec and cosine similarity. For this purpose, we created another Jupyter Notebook named `word2vec.ipynb` to develop the functions.

NOTE: The code is inspired by the code from **Seminar 1**.

Algorithm	PROS	CONS
TF-IDF + Cosine Similarity	Simple to implement and understandable. Fast computation, especially for sparse vectors. Works well for short queries. No need for parameter tuning.	Sensitive to document length. Does not handle term frequency saturation. Assumes linear term importance. Less effective for long documents. Ignores semantic similarity between terms.
BM25	Incorporates term frequency saturation. Adjusts for document length with tunable parameters. Strong baseline in IR tasks. Empirically proven to outperform TF-IDF in many settings. Handles longer queries better than TF-IDF.	Requires tuning. Slightly more computational overhead. Doesn't incorporate semantic or contextual information. Less intuitive than TF-IDF.
Custom	Combines textual and numerical relevance. Highly customizable via parameters. Reflects real-world product priorities. Improves user satisfaction by ranking affordable and popular items higher.	Parameter tuning can introduce bias. Numerical fields may be noisy, missing, or inconsistent. More complex to explain and maintain. Sensitive to outliers in numerical values.

Table 1: Pros and cons for each algorithm

The main steps we need to perform to compute the word2vec ranking with cosine similarity are:

- For all documents in the corpus, separate into **sentences** the products' information (more on what we considered "sentences" below).
- For all sentences, create a **word2vec model** that gets the embedding of all the words in the vocabulary of the corpus.
- Get the embedding of the documents/products by averaging the vector representations of all the words in its sentences.
- Insert a query and get its vector representation in the same way as the documents' representation.
- Compute the cosine similarity between all the documents and the query and get the ranking.

2.1 Process Products

In the function `process_products`, we combine and preprocess the fields into a sentence of preprocessed term for all the documents in the corpus. Given that the Word2Vec approach generates the word embeddings from the context of the words in the sentences, we decided to group all the categorical fields of each product to produce a product description in the following way:

- **Sentence:** title + description + brand + category + subcategory + seller + product_details

At first, we thought of grouping the fields of each product to generate three sentences in the following way:

- **Sentence 1:** title + description
- **Sentence 2:** brand + category + subcategory
- **Sentence 3:** seller + product_details

The reasoning behind it was that we observed that, in most fields, the text is just one or two words (e.g. category, brand). This would give no context for word2vec to generate a good embedding for these words. In our first approach of splitting and concatenating similar fields together in sentences, we wanted single-word fields to get extra context from other fields, generating a better embedding for all the words.

However, the performance left much to be desired. Therefore, we thought of generating a single sentence per product, and this gave us much better results. The final implementation considers one sentence per product. It is also important to note that we decided to use the `preprocess_string` from `gensim.utils` (used in the Seminar 1 code) to preprocess the words because we saw that it removes numbers (which our processing functions do not do). From our first results, we saw that numbers were appearing too often and decided to change the pre-processing function to this one.

2.2 Word2Vec Embedding

To build the word2vec model, we need to put together all the sentences in all the products in a single list. This is what we do in this part of the code:

```
sentences = []

for prod_sentences in product_sentences.values():
    sentences.extend(prod_sentences)
```

Then we train the model as follows:

```
model = Word2Vec(sentences=sentences,
                 vector_size=100,
                 window=7,
                 min_count=5,
                 negative=10,
                 sg=1
)
```

- We decided to use a `vector_size` of 100 because it is a balanced number between having high enough dimensionality (have more information), and not being too computationally expensive.
- We decided on a window of 7 because we first looked at the value of the parameter in seminar 1, but in that case it was based on topics and we do not particularly need a large context to build the word embeddings (some of the sentences are not even 7 words long).
- The `min_count` equal to 5 is the same one used in the seminar. We will only need to take into account later what happens with words that do not have an embedding because they do not appear in the corpus this minimum number of times.
- `negative=10` was chosen based on some of the AI's recommendations and indications (it is lower than the one used in the seminar).
- We also chose to use skip-gram because it is the most used in IR tasks.

From the obtained model (we call it `w2v_model`), we can create the embedding for the products. The function `build_text_terms` returns, for each product, the list of all terms in the document (instead of being split into sentences).

The function `get_embedding` is given a text (list of words) and the trained `w2v_model`, and computes the embedding for all given words.

With these mentioned functions, we then compute the documents/products embeddings as:

```
doc_embeddings = {}
for pid, sentences in product_sentences.items():
    doc_embeddings[pid] = get_embedding(sentences, word2vec_model)
```

The final function needed is the one we named `cosine_similarity`:

```
def cosine_similarity(document_representation, query_representation):
    dot_product = np.dot(document_representation, query_representation)
    norm_document = np.linalg.norm(document_representation)

    if norm_document == 0:
        return 0.0

    return dot_product / (norm_document)
```

It receives the embedding of both a document and a query (`document_representation` and `query_representation`), and returns their cosine similarity. Notice that in our case, we only normalize by the document's vector length because the query's length is a constant for all the documents.

Then, to generate the ranking, we need to obtain for a given query, its representation using `get_embedding` (as with the documents), and compute the cosine similarity for the query with each document and sort in non-increasing order by similarity score. All of this is done in the function `rank_documents` as follows:

```
def rank_documents(w2v_model, doc2vec, query, preprocess=preprocess_string):
    query_terms = preprocess_string(query)
    query_embedding = get_embedding(query_terms, w2v_model)

    sim_scores = []
    for pid, doc_embedding in doc2vec.items():
        score = cosine_similarity(doc_embedding, query_embedding)
        sim_scores.append([score, pid])

    sim_scores.sort(key = lambda x: x[0], reverse=True)

    return sim_scores
```

Notice that we preprocess the query the same way we preprocessed the data before training the model

2.3 Ranking Results

2.4 Better Representations

We know from theory that there are better representations for documents and IR tasks than Word2Vec. For example:

- **Doc2Vec**: it is a document embedding that represents an entire document including its whole context and semantics instead of averaging its individual words' representation. It is able to capture more semantics and context (through word order) in a single document embedding. The results of doing IR tasks like ranking and search with these embeddings are better than using only Word2Vec.
- **Sentence2Vec**: they generate embeddings for an entire sentence instead of a single word or a whole document at once. They are better at capturing sentence semantics. It works best in short texts.
- **Transformer-Based**: embeddings like **Sentence-BERT** that generate the representation of a document through the word's meaning in the whole context. They can capture long-range dependencies and perform very well when computing the semantic similarity.

To summarize the pros and cons of each type of embedding, we created table 2.

Embedding	PROS	CONS
Word2Vec	Efficient, short training time, and faster. Good word embeddings for semantic relationships between them. Good for traditional IR tasks.	Not context aware. Looses sentence and document structure.
Doc2Vec	The dedicated vectors to each document are very meaningful for that document. It captures the word order and works well for long documents. Robust to stop words and noisy text.	Training on the whole corpus (slower). New documents are completely new and require inference.
Sentence2Vec	Better at capturing sentence meaning. Handles word order and syntax. Works well on short texts.	Drop performance on long documents. More computationally expensive.
Transformer-Based	Captures deep contextual meaning. Outperforms the rest of methods on semantic similarity. Works the best for IR tasks.	It has the highest computational cost. The models have a large size.

Table 2: Pros and cons for each algorithm

3 PART 3: Engine

To reproduce the pipeline indicated in the PART 3's document instructions, we reused some of the code from PART 2 and put everything together in the file `ranking.py`. The pipeline consists of:

```

# Load the corpus
json_path = os.getenv("DATA_FILE_PATH")
corpus = corpus_df_loading(json_path)

# Preprocess the products and precompute the inverted index
products = get_or_create(products_filepath, lambda: process_products(corpus))
numeric_data = get_or_create(products_numeric_data_filepath, lambda:
                             get_numerical_info_by_product(corpus))
index, index2title, tf, df, idf = get_index_and_metrics(corpus)
df_bm25, idf_bm25, Ld, Lave = compute_prod_data(products=products)

# ENGINE START
print("\nClothing Articles Search Engine")
exit = False
while(not exit):

    # STEP 1: take a query as input
    query = input("\nEnter search terms: ")

    # STEP 2: filter the products
    query_terms, filtered_prod = filter(query, products=products)

    # STEP 3: rank the filtered products by different algorithms
    # Rank with the three methods
    if (len(filtered_prod) == 0):    # Compute ranking only if we found documents during
                                    filtering
        print("No results!")
    else:
        # TF-IDF + cosine similarity
        # code here to obtain the ranking and print it

        # BM25
        # code here to obtain the ranking and print it

        # Custom Ranking
        # code here to obtain the ranking and print it

    answer = input("Would you like to do another search[Y/n]: ").strip().lower()
    exit = answer in ("N", "n", "no", "No", "NO")
# ENGINE END

```

Notice that we chose to have all the preprocessing step before the engine starts to compute the inverted index, and the data for all the algorithms (all the offline computations). In addition, for the repeated execution of the script, we created the functions `dump_data`, `load_data`, `get_or_create` to store result of the heavy computations in files and load them instead of recomputing in following executions. The while loop makes the engine behave like an infinite query taker.

TF-IDF + cosine similarity

1. [score=2.83] Men Regular Fit Checkered Casual Shirt
2. [score=2.83] Solid Men Henley Neck Green T-Shirt (Pack of 3)
3. [score=2.83] Solid Men Henley Neck Red T-Shirt (Pack of 3)
4. [score=2.78] Full Sleeve Solid Men Leather Jacket
5. [score=2.78] Full Sleeve Solid Men Leather Jacket
6. [score=2.78] Full Sleeve Solid Men Leather Jacket
7. [score=2.78] Full Sleeve Solid Men Leather Jacket
8. [score=2.78] Full Sleeve Solid Men Leather Jacket
9. [score=2.76] Full Sleeve Color Block, Applique Men Leather Jacket
10. [score=2.76] Full Sleeve Color Block, Applique Men Leather Jacket
11. [score=2.71] Full Sleeve Solid Men Leather Jacket
12. [score=2.71] Full Sleeve Solid Men Leather Jacket
13. [score=2.71] Full Sleeve Solid Men Leather Jacket
14. [score=2.67] Full Sleeve Washed Men Denim Jacket
15. [score=2.67] Full Sleeve Washed Men Denim Jacket
16. [score=2.62] Full Sleeve Printed Men Denim Casual Jacket
17. [score=2.58] Full Sleeve Washed Men Denim Jacket
18. [score=2.58] Full Sleeve Washed Men Denim Jacket
19. [score=2.46] Full Sleeve Solid Men Leather Jacket
20. [score=1.96] Full Sleeve Solid Men Jacket
21. [score=1.95] Full Sleeve Solid Men Jacket
22. [score=1.90] Full Sleeve Solid Men Riding Jacket
23. [score=1.90] Full Sleeve Solid Men Riding Jacket
24. [score=1.86] Full Sleeve Solid Men Quilted Jacket

Figure 1: Ranking using TF-IDF and cosine similarity

BM25

1. [score=15.57] Full Sleeve Solid Men Leather Jacket
2. [score=15.57] Full Sleeve Solid Men Leather Jacket
3. [score=15.57] Full Sleeve Solid Men Leather Jacket
4. [score=15.41] Full Sleeve Solid Men Leather Jacket
5. [score=15.41] Full Sleeve Solid Men Leather Jacket
6. [score=15.41] Full Sleeve Solid Men Leather Jacket
7. [score=15.41] Full Sleeve Solid Men Leather Jacket
8. [score=15.41] Full Sleeve Solid Men Leather Jacket
9. [score=14.33] Full Sleeve Solid Men Leather Jacket
10. [score=14.17] Full Sleeve Color Block, Applique Men Leather Jacket
11. [score=14.17] Full Sleeve Color Block, Applique Men Leather Jacket
12. [score=12.67] Full Sleeve Solid Men Quilted Jacket
13. [score=12.54] Full Sleeve Solid Men Jacket
14. [score=12.33] Full Sleeve Solid Men Jacket
15. [score=12.19] Full Sleeve Printed Men Denim Casual Jacket
16. [score=12.13] Full Sleeve Solid Men Riding Jacket
17. [score=12.13] Full Sleeve Solid Men Riding Jacket
18. [score=10.96] Full Sleeve Washed Men Denim Jacket
19. [score=10.96] Full Sleeve Washed Men Denim Jacket
20. [score=10.28] Full Sleeve Washed Men Denim Jacket
21. [score=10.21] Full Sleeve Washed Men Denim Jacket
22. [score=9.35] Solid Men Henley Neck Green T-Shirt (Pack of 3)
23. [score=9.28] Solid Men Henley Neck Red T-Shirt (Pack of 3)
24. [score=8.27] Men Regular Fit Checkered Casual Shirt

Figure 2: Ranking using BM25

Custom Ranking

1. [score=4.86] Full Sleeve Solid Men Leather Jacket
2. [score=4.86] Full Sleeve Solid Men Leather Jacket
3. [score=4.86] Full Sleeve Solid Men Leather Jacket
4. [score=4.82] Full Sleeve Solid Men Leather Jacket
5. [score=4.82] Full Sleeve Solid Men Leather Jacket
6. [score=4.82] Full Sleeve Solid Men Leather Jacket
7. [score=4.82] Full Sleeve Solid Men Leather Jacket
8. [score=4.82] Full Sleeve Solid Men Leather Jacket
9. [score=4.73] Full Sleeve Color Block, Applique Men Leather Jacket
10. [score=4.73] Full Sleeve Color Block, Applique Men Leather Jacket
11. [score=4.45] Full Sleeve Solid Men Leather Jacket
12. [score=4.26] Full Sleeve Solid Men Jacket
13. [score=4.26] Full Sleeve Solid Men Quilted Jacket
14. [score=4.20] Full Sleeve Solid Men Jacket
15. [score=4.11] Full Sleeve Solid Men Riding Jacket
16. [score=4.07] Full Sleeve Solid Men Riding Jacket
17. [score=3.81] Full Sleeve Washed Men Denim Jacket
18. [score=3.60] Full Sleeve Washed Men Denim Jacket
19. [score=3.57] Full Sleeve Washed Men Denim Jacket
20. [score=3.42] Solid Men Henley Neck Green T-Shirt (Pack of 3)
21. [score=3.40] Solid Men Henley Neck Red T-Shirt (Pack of 3)
22. [score=0.04] Full Sleeve Printed Men Denim Casual Jacket
23. [score=0.04] Full Sleeve Washed Men Denim Jacket
24. [score=0.03] Men Regular Fit Checkered Casual Shirt

Figure 3: Ranking using the custom ranking